



UNIVERSITÉ DE ROUEN NORMANDIE
FACULTÉ DES SCIENCES ET TECHNOLOGIES

Mini-projet d'XML version 1.0

Participants:

Abdelhalim ESSEAMI
Mohamed BENGUERAR

M. Stéphane Hérouville

Année Universitaire 2024-2025

Table des matières

1	Introduction	3
1.1	Contexte du projet	3
1.2	Objectifs	3
1.3	Technologies utilisées	3
2	Analyse du format rss25SB	3
2.1	Structure générale	3
2.2	Contraintes et validations	4
2.2.1	Contraintes structurelles	4
3	Architecture du service REST	4
3.1	Vue d'ensemble	4
3.2	Structure des packages	4
4	Implémentation des endpoints	5
4.1	Page d'accueil	5
4.1.1	Spécification	5
4.1.2	Implémentation	5
4.1.3	Template HTML	5
4.2	Page d'aide	6
4.2.1	Spécification	6
4.2.2	Implémentation	6
4.3	Liste synthétique des articles	7
4.3.1	Format XML	7
4.3.2	Format HTML	9
4.4	Détail d'un article	10
4.4.1	Format XML	10
4.4.2	Format HTML	10

1 Introduction

1.1 Contexte du projet

Ce projet s'inscrit dans le cadre du module "Langage Web 2 - XML". Son objectif principal est de développer un service REST complet permettant la gestion de flux RSS conformes au format rss25SB. Au cours des différentes séances précédentes, nous avons élaboré plusieurs composants liés à ce format, tels qu'un schéma XSD (pour la validation du contenu) et une feuille XSLT (pour la génération de pages HTML), que nous utiliserons pour implémenter les différentes fonctionnalités demandées.

1.2 Objectifs

Les objectifs principaux de ce projet sont :

1. Déployer un service REST permettant de gérer les flux RSS25SB
2. Créer un outil client pour envoyer et consommer des flux RSS25SB
3. Mettre en application les connaissances acquises en XML, XSD, XSLT et XQuery
4. Développer une solution RESTful exploitable en conditions réelles

1.3 Technologies utilisées

- **Backend** : Spring Boot (Java)
- **Validation** : Schema XSD (Avant ajout)
- **Transformation** : XSLT (Affichage de page HTML)
- **Base de données** : PostgreSQL
- **Déploiement** : CleverCloud
- **Tests** : Postman

2 Analyse du format rss25SB

2.1 Structure générale

La structure hiérarchique est organisée sur 3 niveaux :

- **Niveau 0** : Déclaration XML et élément racine `feed`
- **Niveau 1** : Métadonnées du flux (titre, date, copyright, liens)
- **Niveau 2** : Articles individuels (`item`)
- **Niveau 3** : informations relatifs aux auteurs / contributeurs

2.2 Contraintes et validations

2.2.1 Contraintes structurelles

- L'ordre des balises de niveau 1 et 2 est imposé
- L'ordre des balises de niveau 3 n'est pas imposé
- Une balise ne peut être présente que si sa balise parente est présente
- Les identifiants doivent être uniques

3 Architecture du service REST

3.1 Vue d'ensemble

Le service REST rss25SB suit une architecture en couches basée sur le pattern MVC (Model-View-Controller) avec Spring Boot. L'architecture se compose des éléments suivants :

- **Contrôleurs REST** : Gestion des endpoints HTTP
- **Services métier** : Logique applicative
- **Couche de persistance** : Gestion des données
- **Validateurs** : pour la validation du flux elle se fait via XSD
- **Transformateurs** : Conversion XML/HTML via XSLT

3.2 Structure des packages

```
1 fr.univrouen.rss25SB/  
2     controller/  
3         HomeController.java  
4         GetController.java  
5         ItemController.java  
6         ItemHtmlController.java  
7     service/  
8         ItemService.java  
9         ValidationService.java  
10        FeedService.java  
11    model/  
12        Feed.java  
13        Item.java  
14        ItemSummaryDTO.java  
15        ItemSummaryList.java  
16        Image.java  
17        Link.java  
18        Category.java  
19        Person.java  
20        Author.java  
21        Contributor.java  
22        LocalDateAdapter.java  
23        Content.java  
24    repository/
```

```
25         FeedRepository.java
26         ItemRepository.java
27     config/
28         JaxbConfig.java
```

4 Implémentation des endpoints

4.1 Page d'accueil

4.1.1 Spécification

- URL : /
- Méthode : GET
- Retour : Page HTML/XHTML valide

4.1.2 Implémentation

```
1 @Controller
2 public class HomeController {
3
4     @GetMapping("/")
5     public String home(Model model) {
6         model.addAttribute("TP08", "RSS25SB Service");
7         model.addAttribute("version", "1.0.0");
8         model.addAttribute("developers", "Mohamed BENGUERAR /
9         Abdelhalim ESSELAMI");
10        return "index";
11    }
12 }
```

4.1.3 Template HTML

```
1 <!DOCTYPE html>
2 <html xmlns:th="http://www.thymeleaf.org">
3 <head>
4     <meta charset="UTF-8">
5     <meta name="viewport" content="width=device-width, initial-scale=1.0">
6     <title>RSS25SB Service</title>
7     <link rel="stylesheet" href="/css/style.css">
8 </head>
9 <body>
10    <header>
11        <h1 th:text="${projectName}">RSS25SB Service</h1>
12    </header>
13    <main>
14        <section class="project-info">
15            <h2>Informations du projet</h2>
16            <ul>
```

```

17         <li><strong>Nom du projet:</strong> <span th:text="${
projectName}">RSS25SB Service</span></li>
18         <li><strong>Version:</strong> <span th:text="${version}"
>1.0.0</span></li>
19         <li><strong>Développeur(s):</strong> <span th:text="${
developers}">Votre Nom</span></li>
20     </ul>
21     <div class="university-logo">
22         
23     </div>
24 </section>
25
26 </main>
27 <footer>
28     <p>&copy; <span th:text="${#dates.format(#dates.createNow(), '
yyyy ')}">2025</span> - Universit de Rouen</p>
29 </footer>
30 </body>
31 </html>

```

Explication : Le contrôleur utilise l'annotation `@Controller` de Spring MVC pour gérer la racine du service ("/). Lorsqu'un utilisateur accède à cette URL, la méthode `home()` est appelée. Celle-ci ajoute au modèle des attributs tels que le nom du projet, sa version et les développeurs, puis retourne la vue `index`, qui sera interprétée par le moteur de template **Thymeleaf**.

La page HTML correspondante (`index.html`) exploite ces données grâce aux expressions `th:text`. Elle affiche dynamiquement les informations du projet ainsi que le logo de l'université. Le rendu est stylisé via une feuille de style externe.

4.2 Page d'aide

4.2.1 Spécification

- **URL :** /help
- **Méthode :** GET
- **Retour :** Page HTML/XHTML avec documentation API

4.2.2 Implémentation

```

1 @Controller
2 public class HomeController {
3
4     @GetMapping("/help")
5     public String help(Model model) {
6         return "help";
7     }
8 }

```

Explication : Ce contrôleur Spring gère les requêtes envoyées à l'URL `/help`. Lorsqu'un utilisateur accède à cette route, la méthode `help()` est appelée et retourne la vue `help.html`. Aucune donnée n'est injectée dans le modèle ici, car la vue se contente d'afficher du contenu statique.

La page `help.html` agit comme une documentation simplifiée de l'API. Elle fournit une liste de liens vers les différents services REST disponibles, ainsi que des exemples d'utilisation pour guider l'utilisateur dans la consommation de l'API `rss25SB`.

4.3 Liste synthétique des articles

4.3.1 Format XML

Spécification

- URL : `/rss25SB/resume/xml`
- Méthode : GET
- Retour : Flux XML avec liste résumée

```
1 @Controller
2 @RequestMapping("/rss25SB")
3 public class ItemController {
4
5     private static final Logger LOGGER = Logger.getLogger(ItemController
6         .class.getName());
7     private static final String ATOM_NAMESPACE = "http://www.w3.org
8         /2005/Atom";
9
10
11     @Autowired
12     private FeedRepository feedRepository;
13
14     @Autowired
15     private ValidationService validationService;
16
17     @Autowired
18     private JAXBContext jaxbContext;
19
20     @Autowired
21     private Unmarshaller jaxbUnmarshaller;
22
23     @Autowired
24     private Marshaller jaxbMarshaller;
25
26     private final ItemService itemService;
27
28     public ItemController(ItemService itemService) {
29         this.itemService = itemService;
30     }
31
32     @GetMapping(value = "/resume/xml", produces = MediaType.
33         APPLICATION_XML_VALUE)
34     @ResponseBody
35     public ItemSummaryList getItemSummariesAsXml() {
```

```

32         return new ItemSummaryList(itemService.getAllItemSummaries());
33     }
34 }
35
36 @XmlAccessorType(XmlAccessType.FIELD)
37 public class ItemSummaryDTO {
38
39     @XmlElement
40     private Long id;
41
42     @XmlJavaTypeAdapter(LocalDateTimeAdapter.class)
43     private LocalDateTime publicationDate;
44
45     @XmlElement
46     private String guid;
47
48     public ItemSummaryDTO() {}
49
50     public ItemSummaryDTO(Long id, LocalDateTime publicationDate, String
51     guid) {
52         this.id = id;
53         this.publicationDate = publicationDate;
54         this.guid = guid;
55     }
56 }
57
58 @Service
59 public class ItemService {
60
61     private final ItemRepository itemRepository;
62     private final FeedRepository feedRepository;
63
64     public ItemService(ItemRepository itemRepository, FeedRepository
65     feedRepository) {
66         this.feedRepository = feedRepository;
67         this.itemRepository = itemRepository;
68     }
69
70     public List<ItemSummaryDTO> getAllItemSummaries() {
71         return itemRepository.findAll().stream()
72             .map(item -> {
73                 LocalDateTime date = item.getPublished() != null ? item.
74                 getPublished() : item.getUpdated();
75                 return new ItemSummaryDTO(item.getId(), date, item.
76                 getGuid());
77             })
78             .collect(Collectors.toList());
79     }
80 }

```

Ce contrôleur gère les endpoints REST pour le service `rss25SB`. Il expose la route `/rss25SB/resume/xml` : retourne la liste résumée de tous les articles au format XML. Cette liste est encapsulée dans un objet `ItemSummaryList`.

Ce contrôleur dépend d'un service métier, `ItemService`, injecté via le constructeur.
Service métier : `ItemService`

Le service `ItemService` interagit avec la base de données via le repository `ItemRepository`. Il expose deux méthodes dont une dédiée pour cette route à savoir : `getAllItemSummaries()` : retourne une liste de résumés d'articles sous forme de `ItemSummaryDTO`. Chaque résumé contient l'identifiant, la date de publication (ou de mise à jour), et le guid. `ItemSummaryDTO` Cette classe sert à transférer un sous-ensemble des informations d'un article. Elle est annotée pour pouvoir être sérialisée en XML via JAXB. Elle contient :

- `id` : identifiant unique de l'article.
- `publicationDate` : date de publication ou de mise à jour.
- `guid` : identifiant global de l'article.

La date est adaptée au format XML à l'aide d'un `XmlJavaTypeAdapter`.

4.3.2 Format HTML

Implémentation

Spécification

- **URL** : `/rss25SB/resume/html`
- **Méthode** : GET
- **Retour** : Page HTML avec tableau des articles

```
1
2 @Controller
3 @RequestMapping("/rss25SB")
4 public class ItemHtmlController {
5     private final ItemService itemService;
6
7     public ItemHtmlController(ItemService itemService) {
8         this.itemService = itemService;
9     }
10
11     @GetMapping("/resume/html")
12     public String showItemSummariesHtml(Model model) {
13         model.addAttribute("articles", itemService.getAllItemSummaries())
14     );
15         return "summary";
16     }
17
18 }
19
20 @Service
21 public class ItemService {
22
23     private final ItemRepository itemRepository;
24     private final FeedRepository feedRepository;
25
26     public List<ItemSummaryDTO> getAllItemSummaries() {
27         return itemRepository.findAll().stream()
28             .map(item -> {
```

```

29         LocalDateTime date = item.getPublished() != null ? item.
getPublished() : item.getUpdated();
30         return new ItemSummaryDTO(item.getId(), date, item.
getGuid());
31     })
32     .collect(Collectors.toList());
33 }

```

Ce contrôleur Spring est responsable de l’affichage HTML des articles résumés. Il expose la route : `/rss25SB/resume/html` : lorsqu’un utilisateur accède à cette URL, la méthode `showItemSummariesHtml()` est invoquée. Elle récupère la liste des articles via le `ItemService`, l’ajoute au modèle sous le nom `articles`, puis retourne la vue `summary.html`. Cette vue est ensuite interprétée par le moteur de templates (comme Thymeleaf) pour générer dynamiquement une page HTML contenant la liste des articles.

4.4 Détail d’un article

4.4.1 Format XML

Spécification

- **URL** : `/rss25SB/resume/xml/{id}`
- **Méthode** : GET
- **Retour** : Flux XML avec article complet ou erreur

```

1  @GetMapping(value = "/resume/xml/{id}", produces = MediaType.
APPLICATION_XML_VALUE)
2  public ResponseEntity<?> getItemByIdAsXml(@PathVariable Long id) {
3      Optional<Item> item = itemService.getItemById(id);
4
5      if (item.isPresent()) {
6          return ResponseEntity.ok(item.get());
7      } else {
8          return ResponseEntity.status(HttpStatus.NOT_FOUND)
9              .body(new ErrorResponse(id, "ERROR"));
10     }
11 }

```

Ce contrôleur Spring est responsable de l’affichage HTML des articles résumés. Il expose la route : `/rss25SB/resume/xml/id` : lorsqu’un utilisateur accède à cette URL, la méthode `getItemById()` est invoquée. Elle récupère la l’article en question via le `ItemService`, l’ajoute au modèle sous le nom `item`, puis le flux xml indiquant le succès de l’opération

4.4.2 Format HTML

Implémentation

Spécification

- **URL** : `/rss25SB/html/{id}`
- **Méthode** : GET
- **Retour** : Page HTML avec article complet

Implémentation